Universidad Autónoma del Estado de Hidalgo

REPORTE DE PRÁCTICA NO. 4

ANÁLISIS LÉXICO DE CÓDIGO JAVA

ALUMNO: Samuel Ortiz Salinas

Docente: Dr. Eduardo Cornejo-Velázquez

1. Identificación

- Nombre de la práctica: Análisis léxico de código Java
- No. de práctica: 4
- No. de sesiones: 1
- No. de integrantes máximo por equipo: 2

2. Introducción

Dentro de un compilador, la primera etapa por la que pasa el código fuente es el análisis léxico, que se encarga de leer el programa carácter por carácter y agruparlos en unidades con significado llamadas tokens (por ejemplo palabras reservadas, identificadores, números, operadores, etc) [1]. En prácticas anteriores se trabajó con Autómatas Finitos Deterministas de forma más teórica (tabla de transiciones, diagrama, simulación); en esta práctica se ve que un analizador léxico en realidad es, por dentro, una implementación de esos mismos AFD, nada más que en vez de dibujarlos se escriben como expresiones regulares dentro de un programa. Se construyó un analizador léxico (lexer) para un subconjunto del lenguaje Java, capaz de reconocer palabras reservadas, identificadores, operadores, separadores, literales numéricos, cadenas, caracteres y comentarios, además de reportar errores léxicos cuando aparece un carácter que no corresponde a ningún patrón conocido [3].

3. Marco teórico

Un **token** es un par (tipo, valor) que representa una unidad léxica con significado y que es la salida que el analizador léxico le entrega al analizador sintáctico (parser). Un **lexema** es la cadena de caracteres concreta del código fuente que hizo que se reconociera ese token, por ejemplo `"class"` o `"123"`. Un **patrón** es la regla, casi siempre una expresión regular, que describe qué forma puede tener un lexema de determinado tipo de token [1].
Cada patrón de un lexer corresponde en el fondo a un AFD: la expresión regular se puede convertir en un autómata que va leyendo la cadena símbolo por símbolo y cambia de estado, igual que en las prácticas 2 y 3, solo que aquí el estado de aceptación indica *qué tipo de token se reconoció* [2].
Un punto importante es el **orden de evaluación** de los patrones, porque varios pueden coincidir al mismo tiempo con el mismo texto:

1. Comentarios primero, porque pueden contener casi cualquier cosa adentro.
2. Literales de cadena y de carácter, por la misma razón.
3. Literales numéricos: flotante antes que entero, y hexadecimal/binario/octal antes que decimal (todos empiezan con `0`).
4. Palabras reservadas (`KEYWORD`) antes que identificadores, si no `while` se reconocería como identificador.
5. Operadores compuestos antes que los simples, si no `==` se partiría en dos `=`.
6. Separadores.
7. Espacios en blanco, que se descartan al final (pero sirven para contar líneas).

A esto se le llama el principio de **maximal munch**: el lexer siempre debe de tomar el lexema más largo posible que empate con algún patrón [3].

4. Objetivos

Objetivo general

Construir un analizador léxico en Java, usando expresiones regulares (`java.util.regex`), capaz de reconocer los principales tokens del lenguaje Java (palabras reservadas, identificadores, literales, operadores, separadores y comentarios) y de reportar errores léxicos con su ubicación exacta.

Objetivos específicos

- Distinguir entre token, lexema y patrón usando código Java real.
- Escribir expresiones regulares para cada categoría léxica, respetando el orden de evaluación correcto.
- Implementar el lexer usando un patrón maestro con grupos nombrados (`Pattern`/`Matcher`).
- Detectar y reportar errores léxicos indicando línea y columna.

5. Equipo y software a utilizar

Equipo de cómputo con JDK 17 o superior instalado. Se usó OpenJDK, editado en VS Code y compilado/ejecutado desde la terminal con `javac` y `java`. Para revisar algunas expresiones regulares antes de meterlas al código se usó también regex101.com.

6. Desarrollo

Diseño de la solución

La idea general fue tener un solo patrón "maestro" (`Pattern MASTER`) que junta, con el operador `|` (alternancia), todas las expresiones regulares de cada tipo de token, cada una metida en un grupo con nombre usando la sintaxis `(?<NOMBRE>...)`. Así, en cada vuelta del ciclo el `Matcher` encuentra el siguiente lexema y nada más hay que revisar cuál de los grupos nombrados fue el que hizo match para saber de qué tipo de token se trata.
Para representar un token se usó un `record` (novedad de Java desde la versión 16) en vez de una clase normal, porque un token no necesita más que guardar sus datos (tipo, lexema, línea y columna) sin comportamiento extra, y el `record` ya genera solo el constructor y los getters:

Listing 1: Token como record y enumeración de tipos (JavaLexer.java).

```
enum TokenType {
    JAVADOC, BLOCKCOMMENT, LINECOMMENT,
    STRINGLIT, CHARLIT,
    FLOATLIT, HEXADECIMALLIT, BINARYLIT, OCTALLIT, INTEGERLIT,
    KEYWORD, IDENTIFIER,
    OPERATOR, SEPARATOR,
    WHITESPACE, UNKNOWN
}

record Token(TokenType type, String lexeme, int line, int column) {
    public String toString() {
        return String.format("[L%-3d C%-3d] %-15s -> \"%s\"", line, column, type, lexeme);
    }
}
```

Caso 1: palabras reservadas e identificadores

Lo primero que se hizo fue el patrón de `KEYWORD`, con todas las palabras reservadas de Java separadas por `|` y rodeadas por `\b` (word boundary), y el de `IDENTIFIER`, que es más sencillo: una letra, guion bajo o `$` seguida de letras, dígitos, guion bajo o `$`. Aquí quedó claro por qué el orden importa: como `KEYWORD` se evalúa primero, `while` nunca llega a probarse contra `IDENTIFIER`; y como `KEYWORD` usa `\b`, una palabra como `integer` no se parte en `int` más `eger`, sino que se reconoce completa como `IDENTIFIER`.

Caso 2: operadores y separadores

Aquí apareció el "problema del prefijo": si el patrón de operadores tuviera nada más los símbolos sueltos (`[+*/%<>=!&|^~]`), una cadena como `==` se leería como dos operadores `=` separados, cuando en realidad es uno solo. La solución fue poner los operadores compuestos de 2 y 3 caracteres antes que los simples dentro de la misma alternancia (`==|!=|<=|>=|&&|...|[+*/%<>=!&|^~?:]`), para que el maximal munch tome primero el operador más largo posible.

Caso 3: literales numéricos y de cadena

Para los literales numéricos hubo que tener cuidado con el orden otra vez: como los literales hexadecimal, binario y octal también empiezan con el dígito `0`, se tienen que evaluar antes que el patrón decimal, si no el `0` de `0xFF` se agarraría solo como un `INTEGERLIT` y se quedaría suelto el resto `xFF`. También el patrón de flotante se puso antes que el de entero, porque si no, en `3.14` se reconocería primero el `3` como entero y se quedaría el `.14` sin sentido. Para las cadenas y los caracteres se usó `"([^"\\]|\\.)*"` y `'([^'\\]|\\.)'`, que permiten secuencias de escape como `\n` dentro de la cadena.

Caso 4: comentarios y manejo de errores léxicos

Para los comentarios se agregaron tres patrones: `JAVADOC` (`/**...*/`), que se evalúa antes que `BLOCKCOMMENT` (`/*...*/`) porque si no, un javadoc se reconocería nada más como un comentario de bloque normal; y `LINECOMMENT` (`//...` hasta el final de la línea). En los comentarios de bloque fue importante usar el cuantificador no ávido `*?` en vez de `*`, porque sin él, en un código con dos comentarios de bloque en la misma línea (`/* a */ b /* c */`) el patrón se hubiera comido todo desde el primer `/*` hasta el último `*/`, tragándose por error el código de en medio.

Para los caracteres que no encajan en ningún patrón (por ejemplo un `@` suelto) se usó una estrategia de **recuperación en pánico**: en vez de que el programa se detenga con una excepción, se registra el carácter ilegal en una lista de errores junto con su línea y columna, y el análisis sigue adelante con el resto del archivo.

Listing 2: Deteccion de caracteres ilegales sin detener el analisis (JavaLexer.java).

```
if (m.start() > pos) {
    String illegal = source.substring(pos, m.start());
    for (int i = 0; i < illegal.length(); i++) {
        int col = (pos + i) - lineStart + 1;
        char c = illegal.charAt(i);
        errores.add(String.format(
            "Error lexico [L%d, C%d]: caracter ilegal '%c' (U+%04X)",
            line, col, c, (int) c));
        tokens.add(new Token(TokenType.UNKNOWN, String.valueOf(c), line, col));
    }
}
```

Con esto ya se pudo juntar el patrón `MASTER` final con los 14 tipos de token (comentarios, literales, `KEYWORD`, `IDENTIFIER`, operadores, separadores y espacios en blanco), respetando siempre el orden explicado arriba.

Ejecución y validación

Para probar el lexer se escribió un archivo de prueba `Ejemplo.java` con una clase que usa distintos tipos de literales (decimal, hexadecimal, binario, octal, flotante, cadena y carácter), un ciclo `for`, condicionales `if/else`, un comentario javadoc, un comentario de línea y, al final, un carácter `@` suelto a propósito para forzar el error léxico.

En la Figura 1 se muestra el inicio de la salida obtenida al ejecutar `java JavaLexer Ejemplo.java` desde la terminal, y en la Figura 2 se muestra el final, con los errores léxicos y la tabla de frecuencia de tokens.



Figure 1: Inicio de la salida del lexer sobre Ejemplo.java.

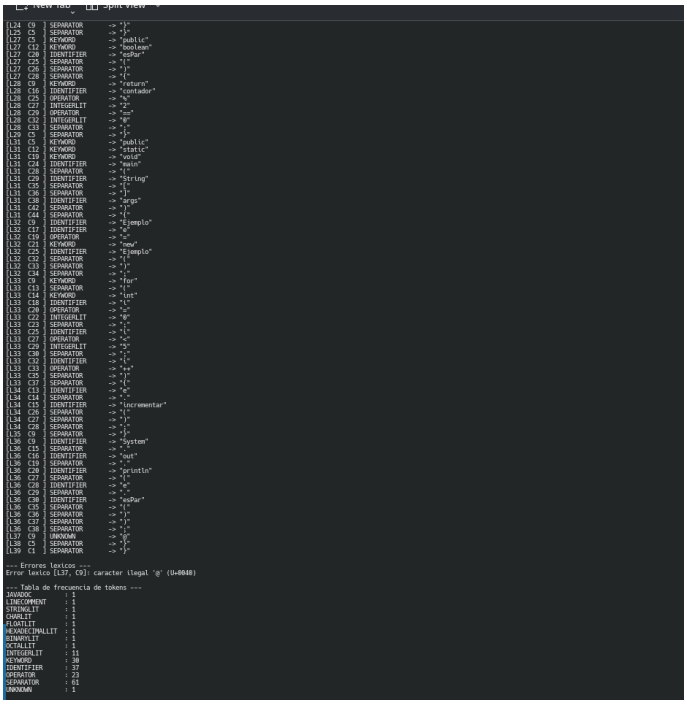


Figure 2: Final de la salida: errores léxicos y tabla de frecuencia.

Un extracto de la salida (se omiten varias líneas intermedias por espacio) fue el siguiente:

Listing 3: Extracto de la salida real del lexer sobre Ejemplo.java.

```
[L1   C1  ] JAVADOC          -> "/**
 * Clase de ejemplo para probar el analizador lexico.
 */"
[L4   C1  ] KEYWORD          -> "public"
[L4   C8  ] KEYWORD          -> "class"
[L4   C14 ] IDENTIFIER       -> "Ejemplo"
[L4   C22 ] SEPARATOR        -> "{"
[L5   C5  ] KEYWORD          -> "private"
[L5   C13 ] KEYWORD          -> "int"
[L5   C17 ] IDENTIFIER       -> "contador"
[L5   C25 ] SEPARATOR        -> ";"
[L6   C31 ] FLOATLIT         -> "3.14159"
[L7   C30 ] STRINGLIT        -> ""Hola\nMundo""
[L8   C28 ] CHARLIT          -> "'A'"
[L9   C30 ] HEXADECIMALLIT   -> "0xFF_AA"
[L10  C30 ] BINARYLIT        -> "0b1010_1010"
[L11  C28 ] OCTALLIT         -> "0777"
    ...
[L18  C9  ] LINECOMMENT      -> "// suma uno al contador"
[L19  C9  ] IDENTIFIER       -> "contador"
[L19  C17 ] OPERATOR         -> "++"
[L20  C22 ] OPERATOR         -> ">="
[L20  C28 ] OPERATOR         -> "&&"
[L22  C46 ] OPERATOR         -> "=="
    ...
[L37  C9  ] UNKNOWN          -> "@"

--- Errores lexicos ---
Error lexico [L37, C9]: caracter ilegal '@' (U+0040)

--- Tabla de frecuencia de tokens ---
JAVADOC          : 1
LINECOMMENT      : 1
```

```
STRINGLIT        : 1
CHARLIT          : 1
FLOATLIT         : 1
HEXADECIMALLIT   : 1
BINARYLIT        : 1
OCTALLIT         : 1
INTEGERLIT       : 11
KEYWORD          : 30
IDENTIFIER       : 37
OPERATOR         : 23
SEPARATOR        : 61
UNKNOWN          : 1
```

Se puede ver que todos los literales numéricos (hexadecimal, binario, octal, flotante y decimal) se reconocieron con su tipo correcto y no se confundieron entre ellos, que el comentario javadoc y el de línea se separaron bien de las palabras reservadas, y que el `@` al final del archivo se detectó como carácter ilegal sin que el programa se detuviera ni dejara de mostrar el resto del reporte.

7. Resultados

Se logró construir un analizador léxico funcional para un subconjunto amplio del lenguaje Java, con 14 tipos de token distintos, usando un solo patrón maestro con expresiones regulares nombradas. Al probarlo sobre el archivo `Ejemplo.java` (39 líneas), se generaron 171 tokens en total (sin contar los espacios en blanco) y se detectó correctamente 1 error léxico (el carácter `@`), sin que el análisis se detuviera antes de terminar de leer el archivo completo.

8. Conclusiones

Esta práctica ayudó a entender que un lexer no es tan distinto de los AFD de las prácticas pasadas, nada más que en vez de dibujarse en papel se escribe directo como expresiones regulares dentro del código, y que la parte más delicada no es tanto escribir cada patrón por separado sino el orden en el que se van probando uno contra otro (comentarios y cadenas primero, keywords antes que identificadores, operadores largos antes que cortos). También quedó más claro por qué conviene que el lexer no se detenga en el primer error: al usar la recuperación en pánico se pudo seguir leyendo el archivo completo y reportar el problema al final, en vez de cortar el análisis a medias.

9. Cuestionario

1. **¿Qué pasa si eliminas los `\b` del patrón KEYWORD?**

   Creo que ya no respetaría las palabras completas. O sea que en `integer` agarraría el `int` de adentro como KEYWORD y dejaría el `eger` suelto, cuando en realidad todo junto debería ser un solo IDENTIFIER.

2. **¿Por qué el patrón KEYWORD debe evaluarse antes que IDENTIFIER?**

   Porque una palabra como `class` también le entra al patrón de IDENTIFIER (nomás son letras). Entonces si se revisa IDENTIFIER primero se la gana y ya nunca se marca como KEYWORD.

3. **¿Podrías construir una sola ER que reconozca ambos tokens a la vez?**

   Creo que sí se podría juntar en una sola ER, pero luego ya no sabría si lo que hizo match fue KEYWORD o IDENTIFIER porque quedarían revueltos en el mismo grupo. Mejor dejarlos separados aunque se repita un poco de patrón.

4. **¿Cómo manejarías el operador ternario `?:` en tu analizador?**

   En mi lexer el `?` y el `:` quedan como dos operadores separados, no como un solo token `?:`. Creo que está bien así porque en Java se escriben separados (`cond ? a : b`), no pegados.

5. **El operador `>>>` (shift sin signo) debe reconocerse antes que `>>`. ¿Por qué?**

   Por el mismo problema del prefijo de antes: si `>>` va primero, en `a >>> b` agarraría dos `>` como `>>` y dejaría un `>` suelto, en vez de reconocer los tres juntos.

6. **¿Qué diferencia hay entre un separador y un operador desde la perspectiva de la gramática?**

   Creo que el operador hace algo con los valores (como sumar o comparar), y el separador nomás marca dónde empieza o termina algo, como los paréntesis o el punto y coma, sin hacer ninguna operación.

7. **¿Por qué `08` NO es un octal válido en Java? ¿Cómo ajustarías la ER?**

   Porque en octal solo existen los dígitos del 0 al 7, y el 8 no es válido ahí. Mi patrón `0[0-7]+[lL]?` ya lo cubre porque después del `0` solo deja pasar dígitos hasta el 7.

8. **¿Cómo extenderías el patrón STRING para soportar Text Blocks de Java 15+ (`"""..."""`)?**

   Pondría un patrón nuevo tipo `"""[\s\S]*?"""` y lo pondría antes que el STRINGLIT normal, porque si no las tres comillas se leerían mal (como cadena vacía más una comilla suelta).

9. **¿Tiene sentido que el analizador léxico verifique si un número está en rango (por ejemplo, `int > 2^31`)?**

   Yo creo que no mucho. Eso ya no es forma del lexema sino su valor, y eso me parece que le toca a otra etapa (semántica), no al lexer.

10. **¿Cómo manejaría tu lexer un comentario de bloque no cerrado al final del archivo?**

    Creo que mal, porque si nunca aparece el `*/` el patrón no hace match y todo se reportaría como puros caracteres ilegales uno por uno, que no se entiende nada. Habría que agregar algo aparte para avisar "comentario sin cerrar".

11. **¿Deberían los JavaDoc comments producir un token diferente a los block comments?**

    Yo creo que sí, y así lo hice (JAVADOC aparte de BLOCKCOMMENT), porque aunque los dos se descartan igual, el javadoc le sirve a otras herramientas como para generar documentación.

12. **¿Cuál es la diferencia entre recuperación de errores en el lexer vs en el parser?**

    Por lo que entendí, en el lexer es más fácil: si un carácter no encaja, nomás se marca como ilegal y se sigue con el siguiente, como hice yo con el `@`. En el parser debe ser más difícil porque ahí ya se rompió la estructura completa, y supongo que tiene que brincarse hasta el siguiente `;` o algo así para seguir.

Referencias Bibliográficas

References

[1] Aho, A. V., Lam, M. S., Sethi, R., Ullman, J. D. (**2007**). *Compilers: Principles, Techniques, and Tools*. Segunda edición. Editorial Pearson Addison-Wesley. U.S.A.

[2] Hopcroft, J. E., Motwani, R., Ullman, J. D. (**2007**). *Introducción a la teoría de autómatas, lenguajes y computación*. Editorial Pearson Addison-Wesley. Tercera edición. Madrid, España.

[3] Nystrom, R. (**2021**). *Crafting Interpreters*. Genever Benning.